SKILL WEEK 11 — SESSION 21

P NEEHARIKA-2520090233

Task 1 :- To Connect Modules, Validate Interfaces, Test End-to-End Execution, Handle Errors, Improve Reliability, Refactor Components.

```
#ifndef CALCULATOR_H
#define CALCULATOR_H

int add(int a, int b);
int subtract(int a, int b);

#endif
```

```
 GNU nano 7.2                          calculator.c *
#include "calculator.h"

int add(int a, int b) {
    return a + b;
}

int subtract(int a, int b) {
    return a - b;
}
```

```
 GNU nano 7.2                                  main.c *
#include <stdio.h>
#include "calculator.h"

int main() {
    int a = 20;
    int b = 10;

    printf("Module Integration Test\n");
    printf("First number: %d\n", a);
    printf("Second number: %d\n", b);
    printf("Addition: %d\n", add(a, b));
    printf("Subtraction: %d\n", subtract(a, b));
    printf("All module tests completed successfully.\n");

    return 0;
}
```

```
root@HP: ~/skill_week11
root@HP:~# mkdir -p ~/skill_week11
cd ~/skill_week11
root@HP:~/skill_week11# nano calculator.h
root@HP:~/skill_week11# nano calculator.c
root@HP:~/skill_week11# nano main.c
root@HP:~/skill_week11# gcc main.c calculator.c -o module_test
root@HP:~/skill_week11# ./module_test
Module Integration Test
First number: 20
Second number: 10
Addition: 30
Subtraction: 10
All module tests completed successfully.
root@HP:~/skill_week11#
```

Task 2 :- To Detect Runtime Errors, Generate Messages, Log Failures, Handle Invalid Syntax, Recover Gracefully, Test Error
Scenarios.

```
  GNU nano 7.2                    task2_error_handling.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <errno.h>

int main() {
    char input[100];
    char *end;
    long number;

    printf("Runtime Error Handling Demonstration\n");
    printf("Enter an integer: ");

    if (fgets(input, sizeof(input), stdin) == NULL) {
        fprintf(stderr, "Error: Unable to read input.\n");
        return 1;
    }

    errno = 0;
    number = strtol(input, &end, 10);

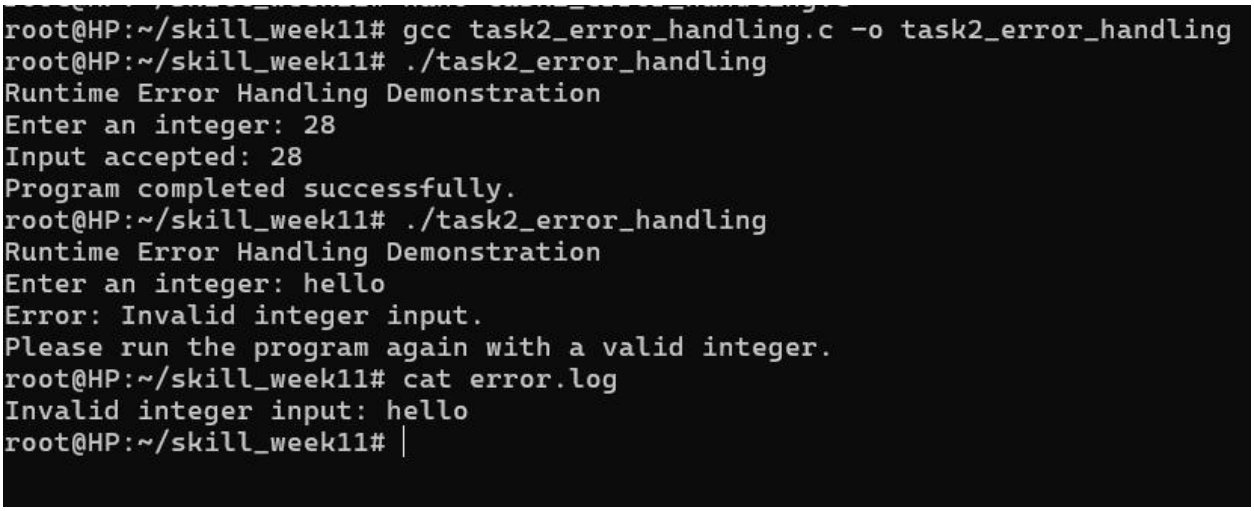
    if (end == input || (*end != '\n' && *end != '\0')
        || errno != 0) {
        fprintf(stderr, "Error: Invalid integer input.\n");

        FILE *log = fopen("error.log", "a");
        if (log != NULL) {
            fprintf(log, "Invalid integer input: %s", input);
            fclose(log);
        }

        printf("Please run the program again with a valid integer.\n");
        return 1;
    }

    printf("Input accepted: %ld\n", number);
    printf("Program completed successfully.\n");

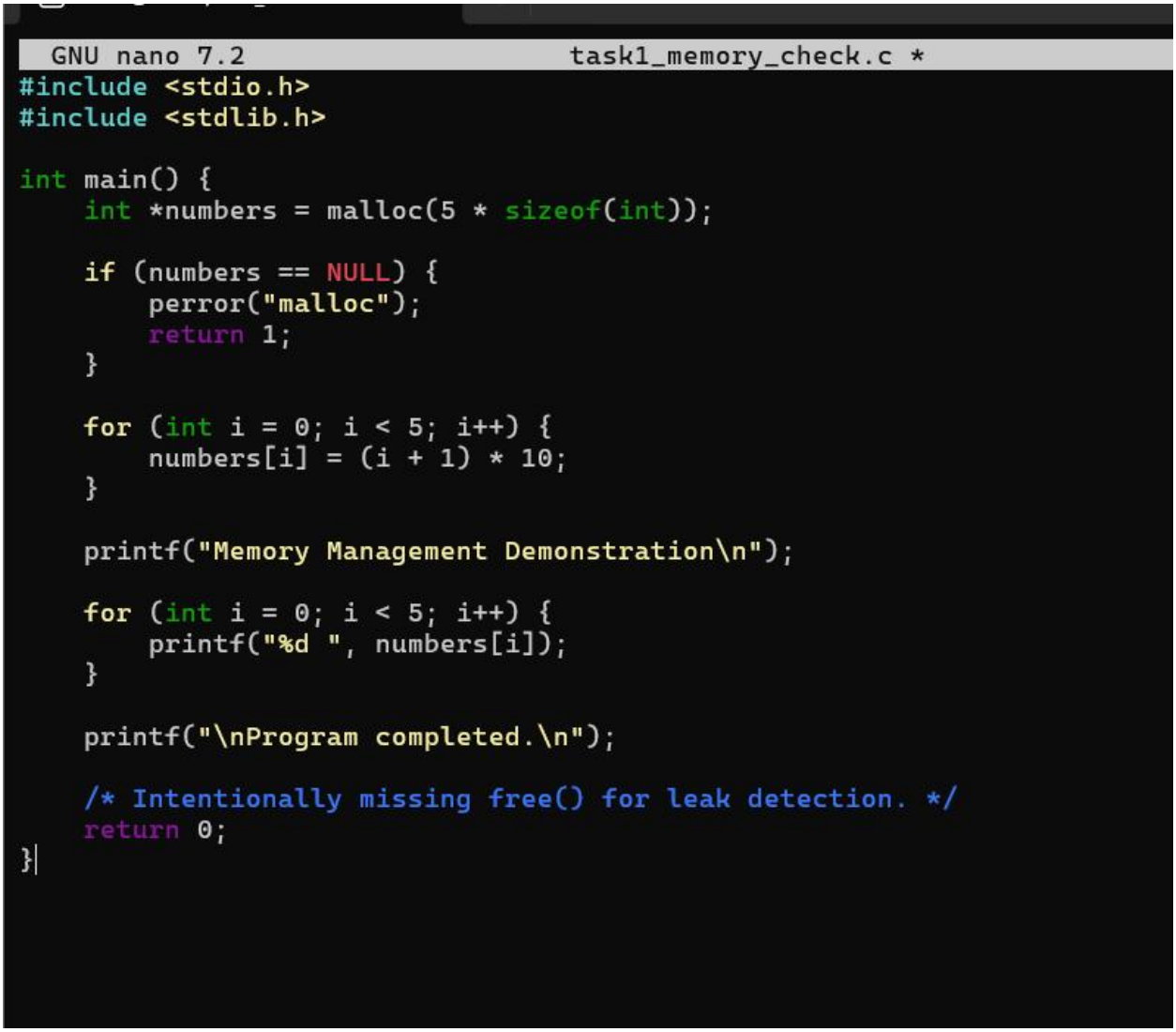
    return 0;
}
```



Observation :- Module integration and runtime error handling were successfully implemented and tested using C programs.

SKILL WEEK 11 — SESSION 22

Task 1 :- To Run Valgrind, Identify Leaks, Fix Memory Errors, Release Resources, Verify Cleanup, Document Findings.



```
  GNU nano 7.2                        task1_memory_check.c *
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *numbers = malloc(5 * sizeof(int));

    if (numbers == NULL) {
        perror("malloc");
        return 1;
    }

    for (int i = 0; i < 5; i++) {
        numbers[i] = (i + 1) * 10;
    }

    printf("Memory Management Demonstration\n");

    for (int i = 0; i < 5; i++) {
        printf("%d ", numbers[i]);
    }

    printf("\nProgram completed.\n");

    /* Intentionally missing free() for leak detection. */
    return 0;
}
```

```
root@HP:~/skill_week11# gcc -g task1_memory_check.c -o task1_memory_check
./task1_memory_check
Memory Management Demonstration
10 20 30 40 50
Program completed.
root@HP:~/skill_week11# valgrind --version
valgrind-3.22.0
root@HP:~/skill_week11# valgrind --leak-check=full ./task1_memory_check
==479== Memcheck, a memory error detector
==479== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==479== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==479== Command: ./task1_memory_check
==479==
Memory Management Demonstration
10 20 30 40 50
Program completed.
==479==
==479== HEAP SUMMARY:
==479==     in use at exit: 20 bytes in 1 blocks
==479==   total heap usage: 2 allocs, 1 frees, 1,044 bytes allocated
==479==
==479== 20 bytes in 1 blocks are definitely lost in loss record 1 of 1
==479==    at 0x4846828: malloc (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-
linux.so)
==479==    by 0x1091BE: main (task1_memory_check.c:5)
==479==
==479== LEAK SUMMARY:
==479==    definitely lost: 20 bytes in 1 blocks
==479==    indirectly lost: 0 bytes in 0 blocks
==479==      possibly lost: 0 bytes in 0 blocks
==479==    still reachable: 0 bytes in 0 blocks
==479==         suppressed: 0 bytes in 0 blocks
==479==
==479== For lists of detected and suppressed errors, rerun with: -s
==479== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
root@HP:~/skill_week11#
```

```
root@HP:~/skill_week11# gcc -g task1_memory_check.c -o task1_memory_check
valgrind --leak-check=full --error-exitcode=1 ./task1_memory_check
==519== Memcheck, a memory error detector
==519== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==519== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==519== Command: ./task1_memory_check
==519==
Memory Management Demonstration
10 20 30 40 50
Program completed.
==519==
==519== HEAP SUMMARY:
==519==     in use at exit: 0 bytes in 0 blocks
==519==   total heap usage: 2 allocs, 2 frees, 1,044 bytes allocated
==519==
==519== All heap blocks were freed -- no leaks are possible
==519==
==519== For lists of detected and suppressed errors, rerun with: -s
==519== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
root@HP:~/skill_week11#
```

Task 2 :- To Design Test Cases, Create Test Scripts, Automate Execution, Verify Outputs, Compare Results, Generate
Reports.

```
  GNU nano 7.2                    task2_test_script.sh *
#!/bin/bash

REPORT="test_report.txt"
PASS=0
FAIL=0

echo "Automated Testing Report" > "$REPORT"
echo "========================" >> "$REPORT"

run_test() {
    name="$1"
    command="$2"
    expected="$3"

    output=$(eval "$command" 2>&1)

    if echo "$output" | grep -Fq "$expected"; then
        echo "PASS: $name" | tee -a "$REPORT"
        PASS=$((PASS + 1))
    else
        echo "FAIL: $name" | tee -a "$REPORT"
        echo "Expected: $expected" >> "$REPORT"
        echo "Actual: $output" >> "$REPORT"
        FAIL=$((FAIL + 1))
    fi
}

run_test "Module addition" "./module_test" "Addition: 30"

run_test "Module subtraction" "./module_test" "Subtraction: 10"

run_test "Valid input" \
    "printf '25\n' | ./task2_error_handling" \
    "Input accepted: 25"
```

```
run_test "Invalid input" \
    "printf 'hello\n' | ./task2_error_handling" \
    "Invalid integer input."

echo "========================" | tee -a "$REPORT"
echo "Passed: $PASS" | tee -a "$REPORT"
echo "Failed: $FAIL" | tee -a "$REPORT"

if [ "$FAIL" -eq 0 ]; then
    echo "All tests passed." | tee -a "$REPORT"
else
    echo "Some tests failed." | tee -a "$REPORT"
fi

root@HP:~/skill_week11# chmod +x task2_test_script.sh
root@HP:~/skill_week11# ./task2_test_script.sh
PASS: Module addition
PASS: Module subtraction
PASS: Valid input
PASS: Invalid input
========================
Passed: 4
Failed: 0
All tests passed.
root@HP:~/skill_week11# cat test_report.txt
Automated Testing Report
========================
PASS: Module addition
PASS: Module subtraction
PASS: Valid input
PASS: Invalid input
========================
Passed: 4
Failed: 0
All tests passed.
root@HP:~/skill_week11#
```

Observation :- Memory leak detection and cleanup were demonstrated using Valgrind, and automated testing was implemented to verify program outputs and generate test reports.